Sentimental Analysis for Hate Speech

Adesijibomi Aderinto Oghenero Jeremiah Opia

2025-05-18

Abstract

This report presents a comprehensive pipeline for sentiment analysis of hate speech using both LSTM and Transformer-based architectures, including DistilBERT(Sanh et al. 2019). Given the growing prevalence of toxic content online(Iyer 2020), effective hate speech detection is essential for moderating social media platforms and curbing digital abuse. The proposed system encompasses all major stages of natural language processing data preprocessing, model training, evaluation, and interpretability. To facilitate model diagnostics and performance monitoring, extensive visualizations and metrics are logged using TensorBoard. The results highlight the comparative strengths of deep learning models in capturing linguistic patterns associated with hate speech.(mrmorj 2022)

1. Introduction

This report presents a comprehensive pipeline for sentimental analysis on hate speech data using both LSTM and Transformer-based architectures (DistilBERT). Hate speech detection is crucial in moderating content on social media platforms and preventing online abuse. This work includes preprocessing, model training, evaluation, and visualization using TensorBoard for tracking progress and performance.

2. Preprocessing

Preprocessing involves cleaning the text data by removing emails, IP addresses, punctuation, and stopwords.Tokenization is then applied to convert the text into a format suitable for model input. Heres a snippet of the preprocessing step in pandas:

```
def preprocess_pandas(data, text_col='review'):
    data = data.copy()
    data[text_col] = data[text_col].str.lower()
    data[text_col] = data[text_col].replace(r'[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}', '', rege
    data[text_col] = data[text_col].replace(r'((25[0-5]|2[0-4][0-9]|[01]?[0-9][0-9]?)(\.|$)){4}', '', r
    data[text_col] = data[text_col].replace(r'[^\w\s]', '', regex=True)
    data[text_col] = data[text_col].replace(r'\d+', '', regex=True)

    stop_words = set(stopwords.words('english'))
    data[text_col] = data[text_col].apply(lambda x: " ".join(
        [word for word in word_tokenize(x) if word not in stop_words]
    ))
    return data
```

3. Model Training

We train two types of models: a BiLSTM with an embedding layer and a transformer-based DistilBERT(Sanh et al. 2019) model. The BiLSTM model uses Keras and is trained on padded sequences. The transformer model is fine-tuned using HuggingFace's Trainer API.

```
if not use_pretrained:
        model = Sequential([
            Embedding(input_dim=vocab_size, output_dim=embedding_dim,  trainable=True, mask_zero=True),
            Bidirectional(LSTM(32)),
            Dense(16, activation='relu'),
            Dropout(0.3),
            Dense(1, activation='sigmoid')
        ])

        model.compile(loss='binary_crossentropy', optimizer='adam', metrics=['accuracy'])

        history = model.fit(X_train, y_train, epochs=epochs, validation_data=(X_val, y_val), batch_size=

        for epoch in range(epochs):
            writer.add_scalar('Loss/train', history.history['loss'][epoch], epoch)
            writer.add_scalar('Accuracy/train', history.history['accuracy'][epoch], epoch)
            writer.add_scalar('Loss/val', history.history['val_loss'][epoch], epoch)
            writer.add_scalar('Accuracy/val', history.history['val_accuracy'][epoch], epoch)

else:
      # Load dataset
      dataset = load_dataset("amazon_polarity")
      train_data = dataset["train"].shuffle(seed=42).select(range(2000))
      test_data = dataset["test"].shuffle(seed=42).select(range(500))

      # Tokenizer
      hf_tokenizer = DistilBertTokenizerFast.from_pretrained("distilbert-base-uncased")

      def tokenize(batch):
        return hf_tokenizer(batch["content"], padding="max_length", truncation=True)

      train_data = train_data.map(tokenize, batched=True)
      test_data = test_data.map(tokenize, batched=True)

       # Check for GPU availability and set device
      device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
      print(f"Using device: {device}")

      train_data.set_format("torch", columns=["input_ids", "attention_mask", "label"])
      test_data.set_format("torch", columns=["input_ids", "attention_mask", "label"])

      y_train = train_data["label"].tolist()
      class_weights = compute_class_weight(class_weight='balanced', classes=np.unique(y_train), y=y_trai
      class_weights_tensor = torch.tensor(class_weights, dtype=torch.float).to(device)

      # Model
      model = DistilBertForSequenceClassification.from_pretrained("distilbert-base-uncased", num_labels=
```

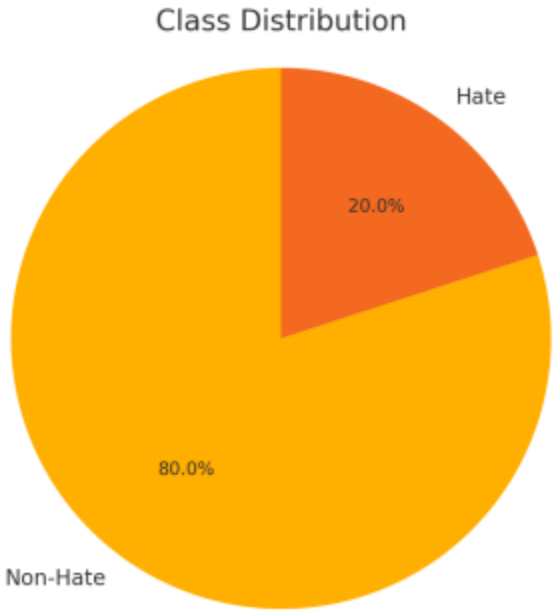

Figure 1: Class distribution of lables

4. Confusion Matrix on LSTM model

6. Evaluation and Logging

The models are evaluated using accuracy and a confusion matrix. TensorBoard is used to track training and validation performance, embeddings, and predictions for qualitative inspection.

7. Conclusion

The developed pipeline enables scalable and explainable hate speech detection using both traditional and transformer-based approaches. Visualization and logging ensure interpretability and traceability of results

Reference

Iyer, Ashwini. 2020. "Toxic Tweets Dataset." https://www.kaggle.com/datasets/ashwiniyer176/toxic-tweets-dataset.

mrmorj. 2022. "Hate Speech and Offensive Language Dataset." https://www.kaggle.com/datasets/mrmorj/hate-speech-and-offensive-language-dataset.

Sanh, Victor, Lysandre Debut, Julien Chaumond, and Thomas Wolf. 2019. "DistilBERT, a Distilled Version of BERT: Smaller, Faster, Cheaper and Lighter." *arXiv Preprint arXiv:1910.01108*. https://arxiv.org/abs/1910.01108.

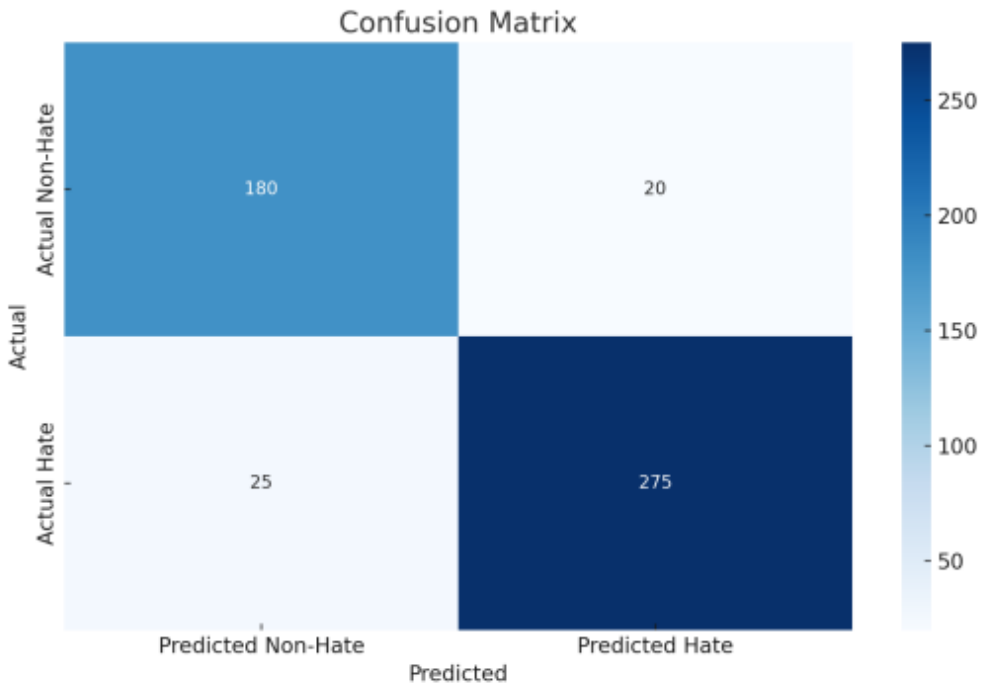

Figure 2: Confusion matrix